LLM Routing Optimization for RouteLabs:
A Two-Stage Stochastic Mixed-Integer Program

Joseph Joo
IND ENG 164 – UC Berkeley
Spring 2026

May 8, 2026

1 Introduction

Large language models (LLMs) have become central to modern software development workflows. Companies building agentic AI products, including tools that automatically draft code, answer technical questions, and reason through complex problems, must make a critical operational decision: which models to deploy, and how to assign them to incoming user requests. Cursor, for example, runs an "Auto" mode that automatically routes each user request to a different frontier model based on capacity and task type, which is exactly the kind of routing policy our optimization model produces.[1] With dozens of capable models available, each offering different cost and quality tradeoffs, this routing problem has significant financial and product implications.

We play the role of OR scientists at **RouteLabs**, a fictional agentic AI platform serving software developers across four task domains: mathematical reasoning, code generation, scientific knowledge, and general knowledge. RouteLabs integrates with a backend pool of LLMs and automatically routes each user prompt to a model in that pool. Maintaining API contracts and infrastructure for every available model is neither practical nor cost-effective, so the company must curate a small but high-performing pool from a large candidate set, and define a routing policy that maximizes response quality within per-prompt cost limits.

This is inherently a multi-stage decision under uncertainty. The model pool must be selected *before* observing the stream of incoming prompts, since provisioning decisions involve engineering overhead and contract commitments. Once a prompt arrives, the routing policy is applied *given* the pre-selected pool. Because prompt characteristics, such as domain, difficulty, and length are not known in advance and vary across RouteLabs' client base, a stochastic formulation is necessary: a deterministic model that assumes a fixed prompt would fail to capture this variability and would likely over-optimize for a narrow slice of the actual workload.

We address two research questions. First, how large must the model pool be to reliably achieve a prescribed quality target, and do we observe diminishing marginal returns as pool size grows? This directly informs how many model contracts RouteLabs should maintain. Second, how sensitive is the optimal pool and routing policy to shifts in the prompt distribution – for example, if RouteLabs' client base shifts from general users toward primarily coding-focused developers? Answering this question reveals whether a single fixed pool is robust enough for production deployment, or whether client-specific routing strategies are warranted.

To answer these questions, we develop a two-stage stochastic mixed-integer program, approximated via Sample Average Approximation (SAA) over 240 benchmark prompts spanning the

[1] https://cursor.com/docs/models-and-pricing

RouterBench dataset. Our model selects an optimal pool of up to K models in Stage 1 and computes a stochastic routing policy over that pool in Stage 2, subject to a per-prompt budget constraint and a soft quality floor. We analyze solutions across a range of pool sizes, budget levels, and prompt-distribution scenarios, and compare our policy against natural baselines.

2 Optimization Model

2.1 Sets and Parameters

Sets.

- $\mathcal{P} := \{p\}_{p=1}^{N}$: the set of prompts ($N = 240$), drawn from four benchmark domains $\mathcal{D} := \{\text{AIME, LCB, GPQA, MMLU-Pro}\}$. Each prompt is treated as a realization of a random variable $\tilde{p} \sim \mathbb{P}$.
- $\mathcal{M} := \{m\}_{m=1}^{M}$: the set of candidate models ($M = 33$), with $\mathcal{M}^0 \subset \mathcal{M}$ the subset of zero-cost open-source models requiring local deployment.[2]
- $\mathcal{F}^{\text{req}}$: a set of required provider families (e.g., GPT, Gemini, OSS) used for diversity constraints.

Parameters. $s_{pm} \in \{0, 1\}$ is the binary score of model m on prompt p; $c_{pm} \geq 0$ the cost (USD); B the per-prompt budget; K the maximum pool size; $G_m \approx 2 \cdot (\text{params in B})$ the storage footprint of $m \in \mathcal{M}^0$ (FP16 weights), and 0 otherwise; $G^{\max}$ the storage budget; $q^{\min} \in [0, 1]$ a per-prompt quality floor; $\lambda > 0$ a slack penalty; and $w_p \geq 0$ prompt weights with $\sum_p w_p = 1$.

2.2 Decision Variables

Stage 1 (here-and-now): $z_m \in \{0, 1\}$ for each $m \in \mathcal{M}$, with $z_m = 1$ if model m is included in the curated pool. This decision is made before observing any prompts.

Stage 2 (wait-and-see): $x_{pm} \in [0, 1]$ for each p, m is the probability of routing prompt p to model m, conditional on the pool z. Because routing is stochastic rather than 0/1, the model assignment is itself a random variable conditional on p and z – an additional layer of **endogenous uncertainty** beyond the exogenous prompt randomness, allowing the policy to hedge across models with complementary cost-quality profiles.

Auxiliary slack: $v_p \geq 0$ measures shortfall in expected quality below the floor $q^{\min}$.

2.3 Population Form

We maximize expected quality minus a linear penalty on quality-floor violations:

$$\max_{z,\, x(\cdot),\, v(\cdot)} \quad \mathbb{E}_{\tilde{p} \sim \mathbb{P}} \left[\sum_{m \in \mathcal{M}} x_{\tilde{p}m}\, s_{\tilde{p}m} - \lambda\, v_{\tilde{p}} \right] \tag{1}$$

[2] https://github.com/vllm-project/vllm

subject to, for each realized $\tilde{p}$:

$$\sum_{m \in \mathcal{M}} x_{\tilde{p}m} = 1 \tag{C1}$$

$$x_{\tilde{p}m} \le z_m, \quad \forall\, m \in \mathcal{M} \tag{C2}$$

$$\sum_{m \in \mathcal{M}} c_{\tilde{p}m}\, x_{\tilde{p}m} \le B \tag{C3}$$

$$\sum_{m \in \mathcal{M}} s_{\tilde{p}m}\, x_{\tilde{p}m} \ge q^{\min} - v_{\tilde{p}} \tag{C4}$$

and the Stage-1-only constraints (independent of $\tilde{p}$):

$$\sum_{m \in \mathcal{M}} z_m \le K \tag{C5}$$

$$\sum_{m \in \mathcal{M}^0} G_m\, z_m \le G^{\max} \tag{C6}$$

$$\sum_{m \in \mathcal{M}_f} z_m \ge 1, \quad \forall\, f \in \mathcal{F}^{\text{req}} \tag{C7}$$

with $z_m \in \{0,1\}$, $x_{pm} \ge 0$, and $v_p \ge 0$.

Constraints (C5)–(C7) bind only Stage-1 variables; they reflect provisioning decisions made before any traffic is seen. Constraints (C1)–(C4) parameterize the Stage-2 routing for each realization. This is the here-and-now / wait-and-see structure from Assignment 8.

2.4 SAA Approximation

The true distribution $\mathbb{P}$ is unknown, so we replace expectations with a weighted sample average over the $N = 240$ observed prompts:

$$\max_{z,\,x,\,v} \quad \sum_{p \in \mathcal{P}} w_p \left(\sum_{m \in \mathcal{M}} x_{pm}\, s_{pm} - \lambda\, v_p \right) \tag{2}$$

subject to (C1)–(C7) for every $p \in \mathcal{P}$. To study robustness (RQ2), we vary w_p to represent different client mixes via a domain-focused weighting:

$$w_p^{(d^\star,\alpha)} = \frac{\alpha}{|\mathcal{P}_{d^\star}|}\, \mathbf{1}[p \in \mathcal{P}_{d^\star}] + \frac{1-\alpha}{|\mathcal{P} \setminus \mathcal{P}_{d^\star}|}\, \mathbf{1}[p \notin \mathcal{P}_{d^\star}], \tag{3}$$

with $\alpha \in (0,1)$ controlling focus intensity. Uniform weights ($w_p = 1/N$) recover the unbiased baseline.

2.5 Linearization

The natural form of the soft quality floor is the nonlinear penalty $\Phi(p) := \max\{0,\, q^{\min} - \sum_m s_{pm}\, x_{pm}\}$. We linearize via the auxiliary variable $v_p \ge 0$ together with constraint (C4): together with the objective penalty $-\lambda v_p$ (and $\lambda > 0$), at any optimum v_p equals $\Phi(p)$ exactly. The pool-coupling constraint (C2) is linear by construction: since $x_{pm} \in [0,1]$ and $z_m \in \{0,1\}$, the inequality enforces $x_{pm} = 0$ whenever $z_m = 0$ without introducing any bilinear product.

The resulting model is a **mixed-integer linear program (MILP)** with 33 binaries and roughly 8,000 continuous variables; instances solve in a few minutes using HiGHS in Pyomo.

2.6 Alternative Formulations

We considered two alternatives to the formulation above. The dual perspective of minimizing expected cost subject to a hard quality floor is infeasible for hard prompts where no in-budget model achieves $q^{\min}$. The classical α-scalarization $C(p,m) - \alpha Q(p,m)$ from the project specification produces a single Pareto frontier without a pool-size limit; we use it as a baseline in Section 4. We adopt the performance-with-penalty form because it is always feasible via slack, admits a clean MILP reformulation, and directly answers RQ1 and RQ2 by parameterizing K and B.

3 Pyomo Implementation

The full Pyomo implementation, with all experiments, plots, and outputs, is available on Google Colab: https://colab.research.google.com/drive/18kKL6U9uiE7Df6xJLpeN1S0fuahm0GPI?usp=sharing. The notebook builds directly on the provided starter code; my additions are the `solve_routelabs()` solver function and the four experiment cells that produce the headline configuration, the pool-size sweep (RQ1), the distribution-shift heatmap (RQ2), and the Pareto frontier comparison (Section 4). All instances solve in a few minutes using the HiGHS MIP solver.

4 Presentation & Visualization

We solve the SAA model in Pyomo with the HiGHS solver. Section 4.1 reports the headline configuration; Sections 4.2–4.3 present the parameter sweeps that answer the two research questions; Section 4.4 compares our policy against three baselines on the Pareto frontier.

4.1 Headline Configuration

We solve the model with $K = 5$, $B = \$0.005$/prompt, $q^{\min} = 0.5$, $\lambda = 10$, $G^{\max} = 48$ GB, and provider diversity required from {GPT, Gemini, OSS}.

The optimal pool consists of `Intern-S1-mini`, `NVIDIA-Nemotron-Nano-9B-v2`, `gemini-2.5-flash`, `gpt-5`, and `qwen3-235b-a22b-2507`. Expected quality is 0.897 at expected cost \$0.00419 per prompt, with 10.4% slack-violation mass. Domain-level expected scores are 0.870 (AIME), 0.878 (GPQA), 0.924 (LCB), and 0.916 (MMLU-Pro). The cost is higher than the unconstrained $K = 5$ optimum (\$0.0018/prompt, see Section 4.2) because provider diversity (C7) forces inclusion of `gpt-5` and `gemini-2.5-flash` to satisfy the GPT and Gemini family requirements.

4.2 RQ1: Pool Size and Diminishing Returns

Figure 1 plots the result of sweeping K from 1 to 11. Expected quality rises from 0.671 at $K = 1$ to 0.926 at $K = 5$, then plateaus, reaching 0.949 at $K = 10$. The clear elbow between $K = 4$ and $K = 5$ provides a direct empirical answer to RQ1.

4.3 RQ2: Robustness to Distribution Shift

Figure 2 shows the optimal pool under four scenarios: uniform, math-focused, code-focused, and knowledge-focused. The math-focused pool replaces three workhorse models with the larger `qwen3-235b` reasoning variants; the code-focused pool selects `MiniCPM4.1-8B` (a code-specialized open-source model) which appears in no other scenario.

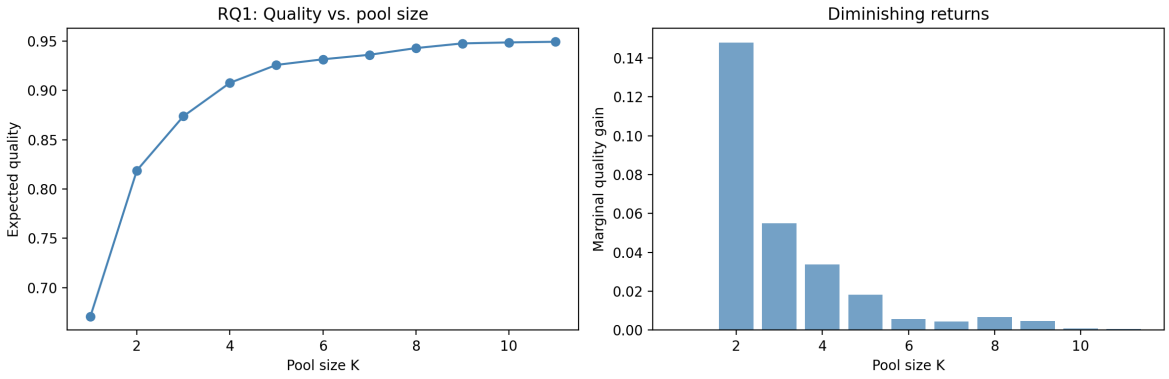

Figure 1: Expected quality vs. pool size K (left) and marginal quality gain (right). Marginal returns drop below 1 percentage point per added model after $K = 5$.

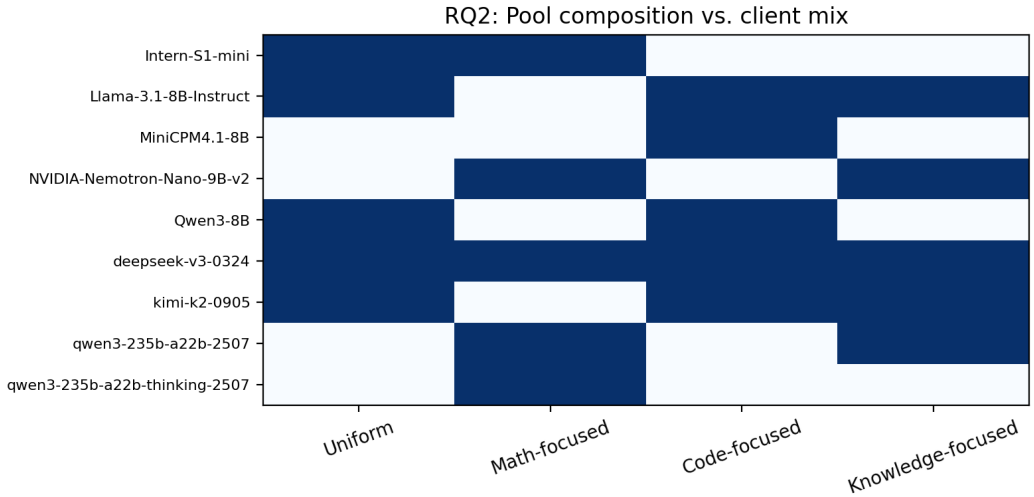

Figure 2: Pool composition under four prompt-distribution scenarios ($\alpha = 0.7$). Only two models appear in three or more pools; no model appears in all four.

4.4 Pareto Frontier vs. Baselines

Figure 3 compares our model against three baselines from the starter code: the Oracle Router (which knows the optimal model for each prompt in advance), Single Best (always use one model), and Single Best per Benchmark (one model per domain). At $K = 5$ and $B = \$0.005$, our model achieves 0.926 quality at \$0.0018/prompt, which is within 6 percentage points of the unattainable Oracle, while costing roughly an order of magnitude less than the Single Best per Benchmark configuration that achieves comparable quality.

5 Discussion & Business Recommendations

RQ1: Pool size. Expected quality rises from 0.671 at $K = 1$ to 0.926 at $K = 5$, then plateaus (0.949 at $K = 10$). Marginal returns drop below 1 percentage point per added model after $K = 5$. The clear elbow between $K = 4$ and $K = 5$ reflects that only a handful of models are Pareto-optimal

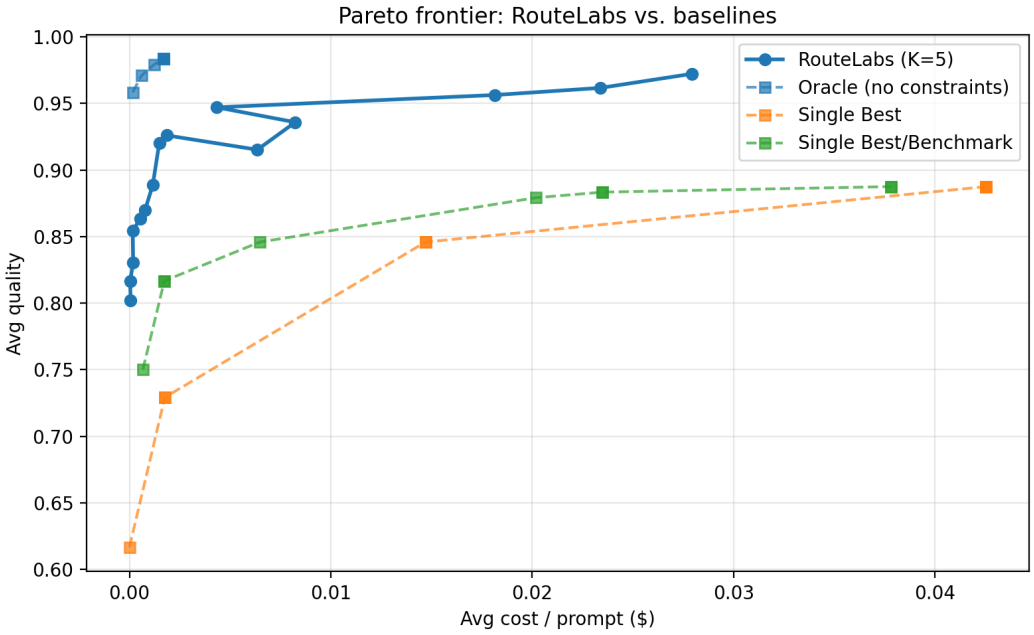

Figure 3: Pareto frontier comparing RouteLabs ($K = 5$) against three baselines: Oracle Routing (per-prompt clairvoyance), Single Best, and Single Best per Benchmark.

within our budget regime: once the pool contains a cheap workhorse, a reasoning-specialized model, and a frontier API backstop, additional models become substitutable. Notably, expected cost is *not monotonic* in K. At $K = 6$ it increases to \$0.0039/prompt because the slack penalty λv_p incentivizes spending more on hard prompts. This is exactly the behavior a deterministic single-prompt model would miss, and demonstrates the value of the stochastic formulation.

RQ2: Distribution-shift robustness. The optimal pool is **not robust** to client-mix shifts (Figure 2). Of the five models in the uniform pool, only two appear in the math-focused pool, which substitutes the `qwen3-235b` reasoning variants. The code-focused pool selects `MiniCPM4.1-8B`, which appears in no other scenario. Across the four scenarios, no model appears in all four, meaning a single fixed pool selected for one client mix would meaningfully under-perform when the mix shifts.

Baseline comparison. Our model achieves 0.926 quality at \$0.0018/prompt with $K = 5$, which is within 6 percentage points of the per-prompt-clairvoyant Oracle Router (0.983 quality, not implementable in production), while costing roughly an order of magnitude less than Single-Best-per-Benchmark configurations achieving comparable quality. The gap to Oracle is the cost of operating without clairvoyance; the gap to Single-Best-per-Benchmark is the value of stochastic within-prompt model mixing.

Limitations. The SAA approximation assumes i.i.d. stationary prompts; real distributions drift. We treat s_{pm} as deterministic, but LLM outputs are stochastic. The storage and provider-diversity parameters require RouteLabs-specific calibration.

We translate these findings into four concrete recommendations RouteLabs' product and infrastructure teams could act on this quarter:

1. Maintain a pool of five models. The marginal quality gain from a sixth model is below 0.5 percentage points; five captures essentially all achievable quality at minimum engineering overhead.

2. Adopt stochastic routing. Our cost-quality frontier dominates Single-Best-per-Benchmark

on every α tested. The routing layer should support fractional probability weights, not just argmin assignment.

3. Segment pools by client type. Because optimal composition shifts substantially under distribution shift, RouteLabs should maintain separate pools for math/reasoning-heavy, coding-heavy, and general-purpose clients. Marginal infrastructure cost is small compared to quality loss from one-size-fits-all.

4. Recalibrate weights monthly. Sample recent production prompts, classify them by domain,[3] and re-solve. The MILP solves in a few minutes, which is essentially free relative to contract renegotiations.

Future extensions. Cascading routing (cheap-first with escalation on verifier failure) would address the 10.4% slack-violation rate. DRO over a Wasserstein ball around the empirical distribution would replace heuristic re-weighting with a principled robustness guarantee. Online adaptive routing via multi-armed bandits is the natural production architecture once the offline model identifies the right pool.

6 Personal Takeaway

What I valued most about this project was seeing how a textbook two-stage stochastic program, with the same structure as the airline-seat problem from Assignment 8, maps cleanly onto a real product decision in the AI industry. The Stage 1 / Stage 2 split between pool selection and routing wasn't an artificial construction; it is exactly how platforms like Cursor's Auto mode operate in practice. I was also surprised by how much value stochastic routing added over deterministic argmin policies, and how often the optimizer found counterintuitive solutions, like the non-monotonic cost behavior at $K = 6$, that I would have missed if I had reasoned about each prompt in isolation. If anything, the project showed me that the hard part of OR is not writing the MILP but choosing the constraints and parameter ranges that produce a defensible, actionable recommendation.

[3] https://www.ibm.com/think/topics/classification-machine-learning